

O Problema de Josephus: Análise e Soluções sob Diferentes Paradigmas

Lucas Blanger, URI - Erechim

Resumo

Este artigo apresenta um estudo progressivo do problema de Josephus, clássico da programação competitiva. São discutidas quatro abordagens: simulação com lista ligada circular em $\mathcal{O}(nk)$, recorrência matemática em $\mathcal{O}(n)$, busca do k -ésimo elemento com *segment tree* em $\mathcal{O}(n \log n)$, e variantes do problema original. Para cada abordagem são apresentados fundamentos, implementação e cenários de aplicação em contest.

1 Introdução

O problema de Josephus tem origem em um episódio histórico relatado por Flávio Josefo, historiador judeu do século I d.C. Durante a Guerra Judaico-Romana, Josefo e seus companheiros decidiram se suicidar em ordem circular com intervalos fixos. Josefo teria calculado rapidamente qual posição ocupar para ser o último a sobrar e, assim, se render aos romanos [4]. Esse episódio deu nome a um dos problemas combinatórios mais clássicos da ciência da computação: dados n pessoas em círculo, elimina-se repetidamente a k -ésima pessoa — qual é a posição da última sobrevivente?

Apesar do enunciado simples, o problema admite abordagens muito distintas, desde simulações diretas até recorrências matemáticas e estruturas de dados sofisticadas. Este artigo apresenta um estudo progressivo dessas abordagens, de $\mathcal{O}(nk)$ até $\mathcal{O}(n \log n)$, discutindo quando cada uma é mais adequada no contexto da programação competitiva.

2 Simulação — $\mathcal{O}(nk)$

A abordagem mais direta para o problema de Josephus é a simulação do processo de eliminação. Representam-se as n pessoas em uma lista circular e percorre-se k posições a cada passo, removendo o elemento encontrado até que reste apenas um.

Uma implementação natural utiliza uma lista ligada circular, estrutura apontada por Tonin [5] como adequada para simulações onde o sentido ou o tamanho do salto pode variar. Cada nodo armazena o número da pessoa e um ponteiro para o próximo elemento. A remoção de um nodo consiste em redirecionar o ponteiro do nodo anterior:

```
1 struct Pessoa {
2     int num;
3     Pessoa *prox;
4     Pessoa(int x, Pessoa *p) : num(x), prox(p) {}
5 };
6
7 int josephus_lista(int n, int k) {
8     Pessoa *b = new Pessoa(1, nullptr);
```

```
9     b->prox = b;
10    for (int i = 2; i <= n; i++)
11        b = (b->prox = new Pessoa(i, b->prox));
12    while (b != b->prox) {
13        for (int i = 1; i < k; i++)
14            b = b->prox;
15        Pessoa *a = b->prox;
16        b->prox = a->prox;
17        delete a;
18    }
19    return b->num;
20 }
```

Código 1: Simulação com lista ligada circular

Cada eliminação exige percorrer k posições, resultando em complexidade $\mathcal{O}(nk)$. Para $k = 1$ ou n pequeno esta abordagem é suficiente; para instâncias maiores, soluções mais eficientes são necessárias.

3 Recorrência Matemática — $\mathcal{O}(n)$

A simulação, embora intuitiva, torna-se inviável para valores grandes de n e k . Uma alternativa elegante é a solução por recorrência matemática, cuja complexidade é $\mathcal{O}(n)$.

A ideia central é observar que, após a primeira eliminação, o problema se reduz a uma instância menor com $n - 1$ pessoas. Seja $J(n, k)$ a posição (indexada em zero) do sobrevivente em um círculo de n pessoas com passo k . A recorrência é:

$$J(n, k) = \begin{cases} 0 & \text{se } n = 1 \\ (J(n - 1, k) + k) \bmod n & \text{se } n > 1 \end{cases} \quad (1)$$

A intuição por trás da fórmula é a seguinte: depois de eliminar a k -ésima pessoa, o círculo é renumerado e o problema recomeça com $n - 1$ participantes. O termo $+k$ corrige o deslocamento causado pela eliminação, e o $\bmod n$ garante a circularidade. Como apontado por Tonin [5], esta fórmula está disponível na literatura clássica [3] e resolve diretamente os problemas Beecrowd 1030, 1031 e 1032.

A implementação recursiva é direta:

```
1 // Retorna posicao 0-indexada do sobrevivente
2 int josephus(int n, int k) {
3     if (n == 1) return 0;
4     return (josephus(n - 1, k) + k) % n;
5 }
6
7 int main() {
8     int n, k;
9     cin >> n >> k;
10    // +1 para converter para indexacao 1-baseada
11    cout << josephus(n, k) + 1 << endl;
12 }
```

Código 2: Recorrência de Josephus

Vale destacar que a versão recursiva pode ser convertida facilmente para iterativa, eliminando o custo de pilha de chamadas e tornando a solução mais robusta para n grande:

```
1 int josephus_iter(int n, int k) {
2     int pos = 0;
3     for (int i = 2; i <= n; i++)
4         pos = (pos + k) % i;
5     return pos + 1; // indexacao 1-baseada
6 }
```

Código 3: Versão iterativa

Ambas as versões têm complexidade $\mathcal{O}(n)$ em tempo e a versão iterativa usa $\mathcal{O}(1)$ de memória, contra $\mathcal{O}(n)$ de pilha da versão recursiva.

4 Segment Tree — $\mathcal{O}(n \log n)$

A recorrência resolve eficientemente o problema clássico, mas falha em variantes que exigem recuperar a ordem de eliminação ou operar sobre instâncias dinâmicas. Para esses casos, modela-se o problema como uma sequência de consultas do tipo *encontrar e remover o k -ésimo elemento vivo*, operação que uma *segment tree* de contagem executa em $\mathcal{O}(\log n)$ [2]. Cada folha i vale 1 se a pessoa i está viva e 0 caso contrário; nodos internos armazenam a soma da subárvore. Para encontrar o k -ésimo vivo, desce-se pela árvore comparando k com a soma da subárvore esquerda:

```

1  const int MAXN = 5e5 + 5;
2  int tree[4 * MAXN];
3
4  void build(int v, int l, int r) {
5      if (l == r) { tree[v] = 1; return; }
6      int m = (l + r) / 2;
7      build(2*v, l, m); build(2*v+1, m+1, r);
8      tree[v] = tree[2*v] + tree[2*v+1];
9  }
10 void update(int v, int l, int r, int i) {
11     if (l == r) { tree[v] = 0; return; }
12     int m = (l + r) / 2;
13     if (i <= m) update(2*v, l, m, i);
14     else update(2*v+1, m+1, r, i);
15     tree[v] = tree[2*v] + tree[2*v+1];
16 }
17 int query(int v, int l, int r, int k) {
18     if (l == r) return l;
19     int m = (l + r) / 2;
20     if (tree[2*v] >= k) return query(2*v, l, m, k);
21     return query(2*v+1, m+1, r, k - tree[2*v]);
22 }
23 int josephus_seg(int n, int k) {
24     build(1, 1, n);
25     int cur = 0, vivos = n;
26     for (int i = 0; i < n - 1; i++) {
27         cur = (cur + k - 1) % vivos + 1;
28         int idx = query(1, 1, n, cur);
29         update(1, 1, n, idx);
30         cur = (cur - 1) % (--vivos);
31     }
32     return query(1, 1, n, 1);
33 }

```

Código 4: Segment tree de contagem e simulação de Josephus

Cada uma das n eliminações executa consulta e atualização em $\mathcal{O}(\log n)$, totalizando $\mathcal{O}(n \log n)$. A vantagem é que esta abordagem permite recuperar a ordem completa de eliminação e se adapta naturalmente a variantes.

5 Variantes

O problema de Josephus admite generalizações relevantes no contexto competitivo. Duas variantes merecem destaque.

Josephus duplo. Dois contadores percorrem o círculo simultaneamente em sentidos opostos, eliminando cada um o k -ésimo e o m -ésimo elemento a partir de suas posições. Quando ambos apontam para a mesma pessoa, ela é enviada para uma fila especial. A recorrência clássica não se aplica diretamente; a solução mais natural usa uma lista ligada circular, conforme discutido na Seção 2, com dois ponteiros independentes [1].

Josephus com pesos. Cada pessoa i suporta w_i eliminações antes de ser removida definitivamente. O problema reduz-se a encontrar repetidamente o k -ésimo elemento com peso restante positivo, operação que a *segment tree* da Seção 4 suporta com uma pequena modificação: cada folha armazena w_i ao invés de 0 ou 1, e a consulta do k -ésimo passa a considerar os pesos acumulados.

Em ambos os casos, a escolha da estrutura de dados segue o mesmo princípio: quando o salto é fixo e apenas o sobrevivente importa, a recorrência é suficiente; quando a ordem de eliminação ou variações dinâmicas são necessárias, a *segment tree* é a ferramenta adequada.

6 Conclusão

Este artigo apresentou um estudo progressivo do problema de Josephus, cobrindo quatro abordagens de complexidades distintas. A simulação com lista ligada circular, embora intuitiva e simples de implementar, torna-se inviável para entradas grandes. A recorrência matemática resolve o caso clássico em $\mathcal{O}(n)$ com código mínimo, sendo a escolha natural em competições quando apenas o sobrevivente é requerido. A *segment tree* de contagem eleva a solução a $\mathcal{O}(n \log n)$, mas habilita consultas mais ricas, como a ordem completa de eliminação e adaptações para variantes do problema.

A progressão das abordagens ilustra um princípio fundamental da programação competitiva: compreender os limites de cada solução é tão importante quanto implementá-la corretamente.

Referências

- [1] Uva 10940 – throwing cards away ii. Online Judge – UVA.
- [2] cp algorithms. Segment tree, 2024. Acesso em: maio 2025.
- [3] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley, 2 edition, 1994.
- [4] Flavius Josephus. *The Jewish War*. 75. Translated by G. A. Williamson. Penguin Classics, 1981.
- [5] Neilor Tonin. Estrutura de dados – parte 1: Utilizando o portal beecrowd, 2009. Apostila de curso, URI – Campus de Erechim.